

TROC Business Case

James Brakefield

The TROC family architecture is a refinement on the RISC architecture, herein referred to as “Register Oriented Computer. The T is for “tagged”. Each register in the register file has a type tag and additional tag bits for floating-point use. The TROC family includes 16, 24 and 32-bit instructions with the 24-bit instructions dominate. There are 32 registers, with register designators using five bits. The 16-bit instructions offer two register designators, 24-bit three, and 32-bit four. Thus the ALU instructions are of the form: $S \text{ op } D \Rightarrow D$, $R \text{ op } S \Rightarrow D$ and $\text{op } (R, S, T) \Rightarrow D$. The register field positions are fixed. Little Endian is used. The left most (S, R or T) can be an immediate value, such as a constant operand or address offset. The immediate values encode small values within the register field and larger values follow the instruction. 24-bit instructions can have a 12-bit immediate value within 32-bits total.

So far: RISC architecture with several instruction sizes and provision for variable length immediate values. Hence very good code density. The 16-bit instructions provide 36 $N \text{ op } D \Rightarrow D$ or $S \text{ op } D \Rightarrow D$ instructions. It is a decent set of instructions for a small implementation. The 24 and 32-bit instructions number about 44 each. This is a more comprehensive instruction set with binary codes allocated that need not be implemented.

The TROC family supports four data types and four independent data sizes. The Tagged feature helps keep the ISA orthogonal with the basic ALU instructions supporting all four types. Typically one wants unsigned, signed, floating-point and a TBD fourth type such as a second floating-point format or fixed point logarithms. Automatic type coercion on mixed type operations is not required. The type tag is predominately a means to provide additional functionality without increased instruction size. The load instructions (sixteen of them) encode both the type and size as they are not stored in memory. Except instructions are provided to load and store a register and its tag information as part of subroutine frame data.

A unique feature is the designation of one register as the “residue” register. It captures otherwise lost information from ALU operations. This includes carry, overflow, upper half of multiply and the remainder of divide. Other methods used are status bits and coprocessor registers, neither of which have elegance or generality.

The PC is also part of the register file as this removes the need for several PC related instructions. For a single cycle implementation it is convenient. On a performance implementation issues arise. And in general performance implementations prefer word aligned instructions and immediate values. There are mechanisms to allow code generation to maintain performance optimal instruction and immediate value alignment.